

OOP

目录

OOP	1
○ 哇好神秘还有绪论课 2026-02-28	1
壹 编译源程序 2026-03-03	2
一、编译链接	2
二、extern 关键字	2
三、宏定义/常量	2
四、Make	3
五、函数重载：一个函数名字，两个以上实现方法。	3
六、auto, decltype	4
auto	4
decltype	4
七、内存申请和释放	4
贰 对象 2026-03-10	4
一、访问权限：	5
二、this 指针	5
三、内联函数	5
四、new delete	6
叁 构造函数和析构函数 2026-03-17	6
一、构造函数：诞生	6
委派构造函数：调用其他构造函数。	6
就地初始化：	6
默认/缺省构造函数：	7
对象数组的初始化：	7
二、析构函数：死亡	7
三、局部对象的构造和析构	8
四、全局对象的构造和析构	8
五、引用	8
六、运算符重载	9
前缀后缀重载区分	9
() 重载	10
[] 重载	10
流运算符重载	10
七、友元 2026-03-24	10
肆 静态变量和静态函数 常量	12
一、静态变量，静态函数	12
静态数据成员 / 类变量	12
静态成员函数	12
二、常量	12
常量数据成员	12
常量成员函数	13
常量对象	13
常量静态变量	13
再谈常量对象和静态对象：构造和析构	13
伍 参数对象的构造和析构	14

○ 哇好神秘还有绪论课 2026-02-28

我去，居然是雷霆方法论基础？！结构化？！面向对象？！神秘秘神？！

不能反直觉反常识？！强强！？ 对象？我的对象在哪里？！弱弱,,, **属性和操作？！静态和动态？！抽象？！**
过程抽象和数据抽象？！eg.类型是什么？！**数据的存储和表示+数据支持的操作？！**

壹 编译源程序 2026-03-03

一、编译链接

举例：argv argc 等参数的运用

实际意义上的一个完整编译流程如下：

```
g++ -c ex1.cpp
g++ -o ex1 ex1.o
```

第一步是编译，第二步是链接。注意到还有多文件链接。

```
g++ -c ex1.cpp ex1.o
g++ -c func1.cpp func1.o
g++ ex1.o func1.o -o ex1
```

当然我们平时偷懒是可以的。只是想告诉你这个编译和链接的实际过程，第二步把两个文件里的函数链接了？！

链接文件里面声明和定义不相同会类型报错。 #include 的本质是把代码直接复制下来了。**声明可以多声明，定义只能定义一次。** 声明的时候可以修改只说参数类型，不喊名字。**变量也可以声明或者定义。即使不赋值也可以被定义！** 注意下面的例子：

```
// num.cpp
int a=1;
```

```
// ex1.cpp
int a;
int main(){
    a=1;
    return 0
}
```

这时就炸了。重复定义。**定义=声明+内存分配**

二、extern 关键字

有一点点烧脑。用于全局变量的共享。

```
//num.h
extern int a;
extern int b;
extern int c;
extern int d[10];

//num.cpp
#include<num.h>
a=1;
b=2;
c=3;
```

头文件中只声明，不定义，目的是防止重复定义。

三、宏定义/常量

请使用 const 以保证类型的正确性！对于宏：

```
#define name(value) command_string
#define sqr(x) ((x)*(x))
```

```
sqr(3+2) = 25
```

请注意这个时候括号十分重要。因为不带括号的时候只是把串原封不动地搬运过来。宏定义可以用来防止头文件被反复包含。

```
// Method 1
#ifdef _HEAD_
#define _HEAD
...(如声明/定义)
#endif
// Method 2
#pragma once
```

还可以用来神秘的 debug。

```
#define DEBUG
#ifdef DEBUG
debug();
#endif
```

四、Make

Makefile 编写规则：只编译并链接被修改的文件，其他不管。但是如果头文件被改变了，所有引用了这个头文件的都要重新编译。格式举例：

```
all: main test
main: main.cpp stu.cpp
    g++ -o main main.cpp stu.cpp
test: stu.cpp stu_test.cpp
    g++ -o test stu_test.cpp stu.cpp
clean:
    rm main test

//bash 举例
make all
make test
make main
make clean
```

Make 的更多用法还是自己去查吧。氧气和臭氧优化后不能 gdb。

五、函数重载：一个函数名字，两个以上实现方法。

关键在于参数类型的不同。但是如果出现了参数的缺省，就会产生二义性。

举例：

```
int fun(int a, int b=0);
int fun(int a);
```

会产生二义性。因为当调用 fun(1) 的时候，编译器无法判断是调用第一个函数还是第二个函数。

但是！

```
int fun(int a);
int fun(float a);
```

就不会产生二义性。因为当调用 `fun(1)` 的时候，编译器会选择第一个函数，因为参数类型是 `int`，而不是 `float`。

六、auto, decltype

`auto` 和 `decltype` 都是 C++11 引入的类型推断机制，可以编译器根据上下文自动推断变量的类型。

auto

```
auto x = 5; // x 的类型被推断为 int
auto y = 3.14; // y 的类型被推断为 double
auto z = "Hello"; // z 的类型被推断为 const char*
```

- `auto` 必须在定义时初始化。
- `auto` 不能用于函数参数声明。
- `auto` 不能用如 `sizeof`, `typeid` 等上下文中。
- 在定义模板函数时用于声明依赖模板参数的返回类型：

```
template<typename _Tx, typename _Ty>
void Multiply(_Tx x, _Ty y){
    auto v = x * y; // 返回类型由编译器根据表达式 x * y 推断
    std::cout << v << std::endl;
}
Multiply(3, 4.5); // 输出 13.5, v 的类型被推断为 double
```

decltype

- 重用匿名类型：

```
struct {
    int a;
    double b;
} myStruct;
decltype(myStruct) anotherStruct; // anotherStruct 的类型与 myStruct 相同
```

除此之外还可以用来追踪推导 `auto` 返回的类型，在 C++ 14 后不必要：

```
auto func(int x, int y) -> decltype(x+y) {
    return x + y; // 返回类型被推断为 int
}
```

七、内存申请和释放

```
int* p = new int(10); // 申请一个 int 类型的内存
int *arr = new int[5]; // 申请一个 int 类型的数组
delete p; // 释放单个 int 类型的内存
delete[] arr; // 释放数组内存
```

`nullptr` 是 C++11 引入的空指针常量，表示一个空指针。使用 `nullptr` 可以避免与整数 0 混淆，提高代码的可读性和安全性。

貳 对象 2026-03-10

成员函数必须在类的内部声明，但是可以在类的外部定义（实现）。

举例：

```
class MyClass {
public:
```

```

    void myFunction(); // 成员函数声明
};
void MyClass::myFunction() { // 成员函数定义
    // 函数体
}

```

一、访问权限：

- public: 公共成员，类的外部可以访问。
- private: 私有成员，只有类的内部可以访问。也是默认的访问权限。
- protected: 受保护成员，只有类的内部和派生类可以访问。

可以通过 `object.member` 的方式访问对象的成员函数和成员变量。或者 `pointer->member` 的方式访问指针指向对象的成员函数和成员变量。在类外只能访问 `public` 成员。

神秘：在类的内部可以访问传入的同一个类的对象的 `private` 成员！因为它们属于同一个类。

举例：

```

class MyClass {
private:
    int privateVar;
public:
    void setPrivateVar(int value) {
        privateVar = value; // 访问自己的 private 成员
    }
    void accessOther(MyClass& other) {
        other.privateVar = 10; // 访问同类的其他对象的 private 成员
    }
};

```

二、this 指针

指向当前对象的指针。可以在成员函数内部使用 `this` 来访问当前对象的成员变量和成员函数。

举例：

```

class MyClass {
private:
    int value;
public:
    MyClass(int v) : value(v) {}
    void printValue() {
        cout << "Value: " << this->value << endl;
    }
};

```

错综复杂的类外类内关系……阿弥诺斯，小白手套欸喂，我这不赖，已取餐。

三、内联函数

由于和宏代码是直接拷贝复制代码到指定位置，会产生莫名其妙的边际效应，你永远无法想象到各种优先级问题，即使加了括号也不一定能完全解决。

相比之下，**内联函数**可以执行类型检查，可调试。它的本质是生成和函数等价的表达式。并且**宏定义**无法调用私有成员函数，而内联函数可以。

注意事项：

- 避免大段代码的内联，因为会增加编译时间和可执行文件的大小。
- **避免对包含循环、递归或复杂逻辑的函数使用内联。**
- 避免将声明和定义分开，因为内联函数必须在调用之前定义。
- 定义在类内部的成员函数默认是内联的，但这只是一个建议，编译器可能会选择不内联。

- 一般构造函数、析构造函数通常会被编译器自动内联。

inline 永远只是建议修饰，不是命令。编译器有权拒绝不合理的内联请求。

四、new delete

new 生成一个类对象，并且返回它的指针。

```
A *pA = new A(1);
```

delete 释放一个类对象。

叁 构造函数和析构造函数 2026-03-17

回顾：要对用户定义类型进行严格检查；要隐藏属性；要自动化对象的初始化和清除，做好对象内存分配等等。

一、构造函数：诞生

- 构造函数没有返回值
- 构造函数名与类名相同
- 构造函数可以重载

举例：

```
class Student {
    int ID;
public:
    Student(int id) : ID(id) {}
    Student(int year, int order){
        ID = year * 10000 + order;
    }
};
```

注意！按照声明顺序而不是初始化顺序排列成员！如下面 ID1 不可预测。

```
class Student {
    int ID1;
    int ID2;
public:
    Student(int id) : ID2(id), ID1(ID2) {}
};
```

委派构造函数：调用其他构造函数。

```
class Info{
public:
    Info(int a):a(a){}
    Info(int a,int b):Info(a){b=b;}
    Info(int a,int b,int c):Info(a,b){c=c;}
};
```

就地初始化：

```
class A{
private:
    int a{0};
    double b=2.0;
public:
    A(){}
};
```

默认/缺省构造函数：

```
class A{
    public:
        A(){};
};

A a;
A b=A();
A c();
// 这个时候 c 是一个函数声明，而不是一个对象声明。它可以生成一个对象。

//显式默认构造函数
class A{
    public:
        A()=default;
};

//显式删除默认构造函数
class A{
    public:
        A()=default;
        A(int i){}
        A(char cls)=delete;
};

A a('c');
```

一个实例生成的时候，会先调用成员的构造函数（如果成员没有就默认构造），再构造自己的。

如果你已经定义了构造函数，编译器不会帮你默认构造了！！

对象数组的初始化：

```
A a[50];
// 只有一个参数
A a[3]={1,2,3};
// 多个参数
A a[3]={A(1,2),A(3,4),A(5,6)};
```

二、析构函数：死亡

- 没有参数
- 唯一
- 没有返回值

举例：

```
class MyClass {
    int num;
    int* ID_list;
public:
    MyClass() { // 构造函数
        // 构造函数体
    }
    ~MyClass() { // 析构函数
        // 析构函数体
        if(ID_list) delete[] ID_list; // 释放内存
    }
};
```

析构函数会在对象生命周期结束时自动调用。比如说程序结束的时候：

```
//我们此前 Test 里面创建了一个 Member m

Test t;
int main(){return 0;}
// 输出: ~Test() ~Member()
```

晚构造，先析构！！

析构函数也会自动隐式定义。但是不会删除指针成员，因此可能导致内存泄漏。

三、局部对象的构造和析构

其实没啥好说的，就是看括号里面的东西。

四、全局对象的构造和析构

- 在 main() 之前构造
- 在同一编译单元中，按照定义顺序构造
- 在不同编译单元中顺序不确定
- 在 main() 执行完 return 语句后析构

尽量少用全局对象！ 全局对象之间最好不要有依赖关系，否则析构顺序会出问题。

五、引用

引用是对象的别名，可以理解为指针常量。引用必须在声明时初始化，并且一旦初始化就不能改变。

举例：

```
int a = 5;
int& ref = a; // ref 是 a 的引用
ref = 10; // 修改 ref 也会修改 a
cout << a << endl; // 输出 10
```

二者绑定，改了一个另一个也会变。而且不能解绑。

有些时候和指针很像。

举例：交换两个变量的值

```
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
```

看一个做错的题：

```
class Int {
public:
    int data;
    Int() { data = 1; }
    Int(int i): data(i) {}
};

void func1(Int& a, Int b) {
    a.data += b.data;
}

Int& func2(Int& a, Int b) {
    func1(a, b);
```

```

    Int tmp(a.data + b.data);
    return tmp;
}

int main() {
    Int a, b(3);
    Int& f = func2(a, b);
    cout << a.data << " ";
    cout << f.data << endl;
    return 0;
}

```

初始状态: a.data = 1 (默认构造), b.data = 3 (值构造)。

调用 func2(a, b) 时, Int& a 是 main 中 a 的引用, Int b 是值拷贝。

1. func1(a, b) 执行 a.data += b.data, 即 $1 + 3 = 4$, main 中的 a.data 变为 4。
2. Int tmp(a.data + b.data) 在栈上构造局部变量, tmp.data = $4 + 3 = 7$ 。
3. return tmp 返回 tmp 的引用。

func2 返回类型为 Int&, 而 tmp 是栈帧上的局部变量。函数返回后栈帧销毁, tmp 随之消亡, main 中的 f 成为悬空引用 (dangling reference), 指向已释放的内存。

注意: tmp 并非“返回前被销毁”, 而是函数返回后随栈帧一起消亡。引用被传递出去了, 但其所指向的对象已不复存在, 后续访问是未定义行为。

引用的用途:

- 作为函数参数, 可以避免复制对象, 提高效率。
- 作为函数返回值, 可以返回引用, 而不是复制对象。
- 作为类的成员变量, 可以简化代码, 提高可读性。
- 相对指针, 引用更安全, 不会出现空指针的情况。

六、运算符重载

举例:

```

A& operator+=(A& a){
    data+=a.data;
    return *this;
}

```

可以重载的运算符:

- 算术运算符: +、-、*、/、%
- 关系运算符: ==、!=、<、>、<=、>=
- 逻辑运算符: &&、||、!
- 位运算符: &、|、^、~、<<、>>
- 单目运算符: +、-、*、&、!
- 自增自减运算符: ++、--
- 赋值运算符: =、+=、-=、*=、/=、%=、&=、|=、^=、<<=、>>=
- 空间运算符: new、delete、new[]、delete[]
- 其他运算符: ()、[]、->、,、->*

前缀后缀重载区分

后缀关键是引入了哑元, 哑元可以没有名, 所以可能实际不会被调用。

```

A operator++(int){
    A temp(data);
    ++data;
}

```

```
    return temp;
}
```

() 重载

```
int operator()(int a, int b){
    return a+b;
}
```

[] 重载

```
int& operator[](int i){
    return data[i];
}
```

如果是引用，则可以修改原对象。否则只能读取。

注意：=, [], (), -> 这几个运算符只能重载为成员函数。否则可能会对是否自动合成重载符产生影响。

流运算符重载

```
ostream& operator<<(ostream& os, const A& a){
    os<<a.data;
    return os;
}

istream& operator>>(istream& is, A& a){
    is>>a.data;
    return is;
}
```

此时我们不得不谈到：友元。

七、友元 2026-03-24

```
#include <iostream>
using namespace std;

class Test {
    int id;
public:
    Test(int i) : id(i) { cout << "obj_" << id << " created\n"; }

    friend istream& operator>> (istream& in, Test& dst);
    friend ostream& operator<< (ostream& out, const Test& src);
};

istream& operator>> (istream& in, Test& dst) {
    in >> dst.id;
    return in;
}

ostream& operator<< (ostream& out, const Test& src) {
    out << src.id << endl;
    return out;
}

int main() {
    Test obj(1);
}
```

```

    cout << obj; // operator<<(cout,obj)
    cin >> obj; // operator>>(cin,obj)
    cout << obj;
    return 0;
}

```

友元是这个对象的“朋友”，可以访问这个对象的私有成员 / 保护成员。

举例：

```

class A{
    int data;
    friend B b;
    b.id=data;
}

```

友元函数举例：

```

class A{
    int data;
    friend void func(A a);
    friend void X::foo(A);
    friend X::X(Y),X::~~X();
};
A a;
func(a);

```

友元函数在类外定义，但是可以访问类的私有成员。且在 private,public 声明没有区别。

更详细的例子：

```

class Y; // 前向声明
class X
{
    int data;
    friend void func(X &x, Y &y);
};
class Y
{
    int data;
    friend void func(X &x, Y &y);
};
void func(X &x, Y &y)
{
    cout << x.data << y.data << endl;
}

```

友元类：

```

class Y {};
class A{
    int data;
    friend class Y;
    friend X;
}
class X{};
// X,Y 都可以访问 A 的私有成员

```

注意：

- 友元关系是**非对称的**，即 A 是 B 的友元，B 不是 A 的友元。
- 友元关系**不传递**，即 A 是 B 的友元，B 是 C 的友元，A 不一定是 C 的友元。
- 友元关系**不具继承性**，即 A 是 B 的友元，C 继承自 B，A 不一定是 C 的友元。
- 友元声明不能直接在里面定义了。

肆 静态变量和静态函数 常量

一、静态变量，静态函数

内部可链接，只能初始化一次，而且作用域仅限其声明的文件。**比如**在其他文件被 `extern` 声明后**不**可以在其他文件中使用。

前缀是 `static`。

静态数据成员 / 类变量

在整个类内所有对象共享，初始化时需要加 `static`。应该在 `.h` 文件中声明，在 `.cpp` 文件（实现文件）中定义，防止重复定义。

在实例化所有对象之前就已经分配了静态数据成员的内存空间。

举例：

```
// Test.h
class Test {
public:
    static int count;
    Test();
    ~Test();
};

// Test.cpp
#include "Test.h"
int Test::count = 0;
Test::Test() { count++; }
Test::~~Test() { count--; }

// main.cpp
#include "Test.h"
int main() {
    Test a;
    Test b;
    cout << Test::count << endl; // 输出 2
    cout << a.count << endl; // 输出 2
    return 0;
}
```

静态成员函数

只能访问静态成员变量，不能访问非静态成员变量。因为非静态成员变量是每个对象独有的，而静态成员函数是所有对象共享的。

二、常量

- `const` 修饰的变量不能被修改。
- 修饰引用/指针时，引用/指针不能被修改，指向的内容不能被修改。
- 修饰函数返回值时，函数返回值不能被修改。

常量数据成员

可以在构造函数初始化列表中初始化、就地初始化，但是不能在构造函数体通过赋值初始化。

常量成员函数

不能修改非静态数据成员，即不能改变对象的状态。

常量对象

只能调用常量成员函数，不能调用非常量成员函数。

错误举例：

```
class Student {
    int ID;
    public:
        Student(int id) : ID(id) {}
        int who() const { return ID; } // 常量成员函数
        int Who() {return ID;} // 编译错误，常量函数不能调用非常量成员
};
```

常量静态变量

需要在类外初始化。但是有两个例外：int 和 enum 可以就地初始化。

不存在常量静态函数。静态函数是所有对象共享的，而常量函数只能在常量对象上调用。

```
class Foo{
    static const int a=1; // 可以就地初始化
    static const int b; // 可以在类外初始化
    static const char* cs; // 不可以就地初始化
};
```

哦哦哦，给你看哥哥的大表格！

	静态数据成员	常量数据成员	常量静态数据成员 (除 int enum 外)	常量静态数据成员 (int, enum)
初始化				
就地初始化		✓		✓
初始化列表初始化		✓		
构造函数体内初始化				
类外初始化	✓		✓	✓
访问				
普通成员函数	✓	✓	✓	✓
静态成员函数	✓		✓	✓
常量成员函数	✓	✓	✓	✓
修改				
普通成员函数	✓			
静态成员函数	✓			
常量成员函数	✓			

再谈常量对象和静态对象：构造和析构

- 常量对象的构造和析构时机和普通对象一样。
 - 在 main() 函数开始执行前，全局对象被构造。
 - 在 main() 函数执行结束后，全局对象被析构。
 - 在执行到局部对象的代码前，局部对象被构造。
 - 在局部对象作用域执行结束后，局部对象被析构。
- 静态全局对象的构造和析构时机和普通全局对象一样。

- 函数中的静态对象在函数调用时构造，**离开作用域不析构**。第二次调用函数时，静态对象不会被重新构造，直接使用上一次构造的结果。在 main() 函数结束后，静态对象被析构。
- 类静态对象：类 A 的 对象 a 作为类 B 的静态变量。
 - 在 main() 函数开始执行前初始化。
 - 在 main() 函数结束后析构。
 - 和 B 的构造和析构时机无关。

伍 参数对象的构造和析构

如果传递的是形参

```
class A{
public:
    const char* s;
    A(const char* str) : s(str) {
        cout << s << "A constructing" << endl;
    }
    ~A() {
        cout << s << "A destructing" << endl;
    }
};
```

如果传递的是引用

```
class A{
public:
    const char* s;
    A(const char* &str) : s(str) {
        cout << s << " A constructing" << endl;
    }
    ~A() {
        cout << s << " A destructing" << endl;
    }
};

void func(A a) {
    cout << a.s << "func" << endl;
}

int main() {
    A a("a");
    func(a);
    return 0;
}
```

输出：(构造一次，析构两次?！)

其实是 b 被拷贝构造函数进行了初始化。函数结束，b 被析构。

```
a A constructing
...
a A destructing
a A destructing
```

如果传递传递指针，你很有可能对同一块空间进行了两次释放，从而报错！

所以尽可能使用引用传递，避免拷贝构造函数和析构函数的调用，还可以节省时间开销。

语句	解析	调用次数
A *p = new A;	堆上创建单个对象	1

A p2[10];	对象数组（10 个元素）	10
A p3;	栈上普通对象	1
A *p4[10];	指针数组（不创建对象）	0
总计		12

- **类内初始化**：C++11 允许声明时直接赋值。
- **静态成员函数**：没有 this 指针，不能访问非静态成员。
- **静态数据成员**：必须类外定义，严禁在 .h 中直接定义（防重复定义）。
- **常量静态整型**：static const int 允许类内初始化。

核心警告： new[] 必须配对 delete[]。

$$\text{Actual Start Address} = pA - 4$$

若误用 delete pA:

1. 仅调用第一个元素的析构函数，导致内存泄漏。
2. 释放地址错误（应从 $pA - 4$ 处释放），直接导致程序崩溃。